

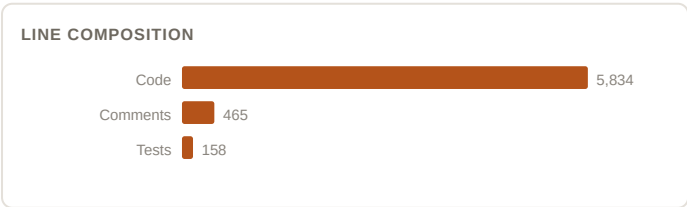
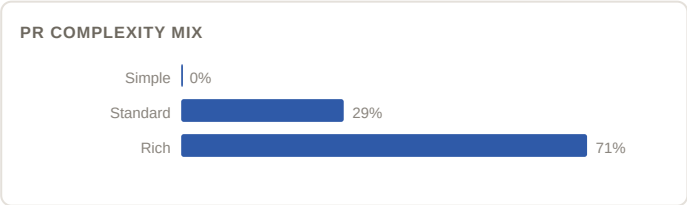
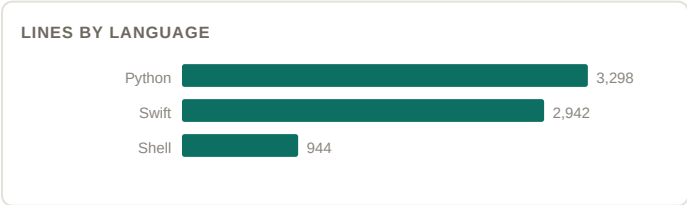
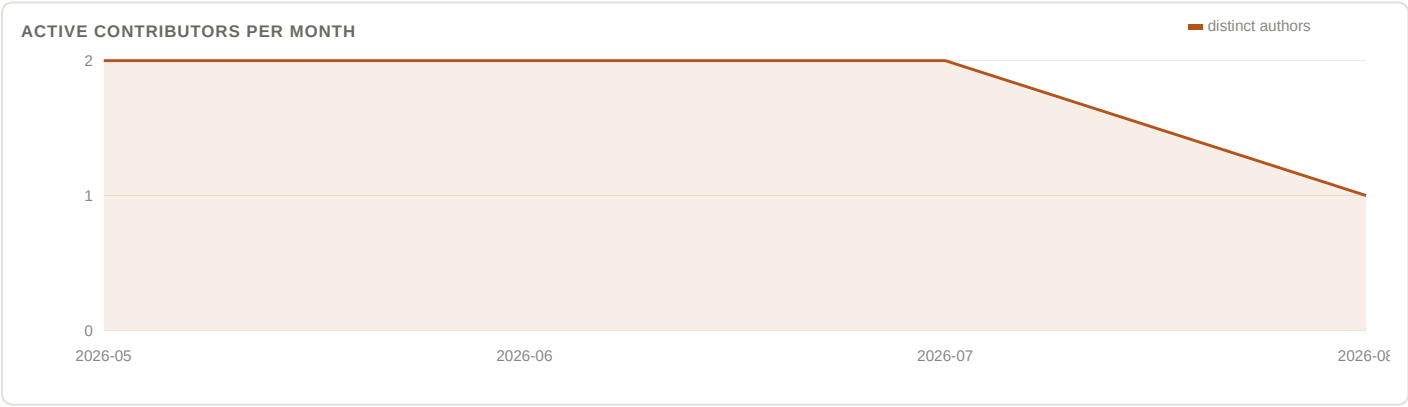
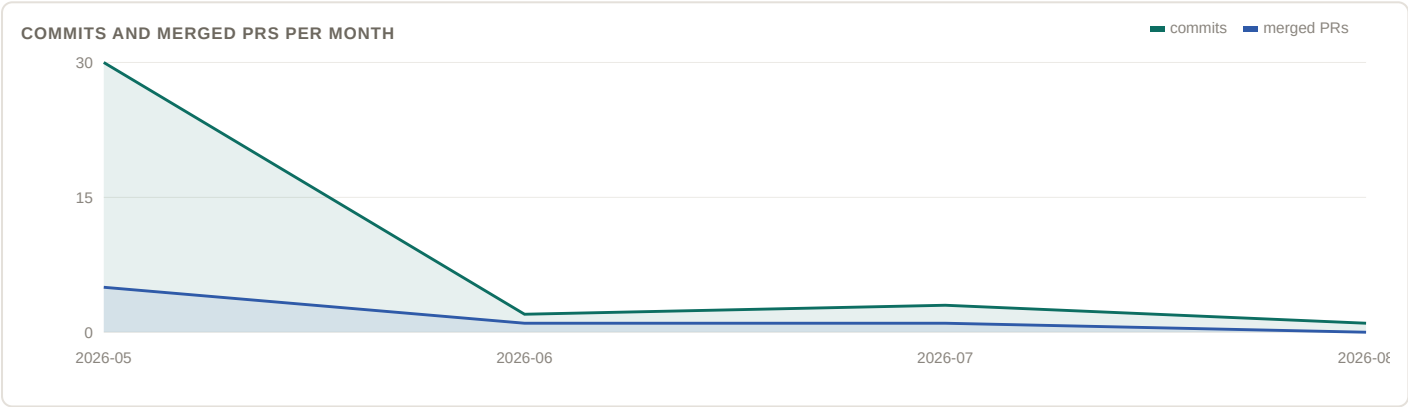
Repository report — TradeSim

Generated 2026-08-13 from the local git checkout.

HIGHLIGHTS

- 7,184 lines across 42 code files. Primary language Python. Mix: Python 46%, Swift 41%, Shell 13%.
- 36 commits by 2 contributors, 2026-05-30 to 2026-08-12.
- 36 commits in the last 12 months (freshness / activity signal).
- 7 merged PRs; sampled mix 0.0% simple / 28.6% standard / 71.4% rich.
- 100.0% of sampled PRs reference an issue in the commit body.
- Test-to-code ratio 2.2% (1 test files); ~100.0% of source files have no matching test.
- Build: CI configured (.github/workflows).
- License: copyleft.
- ~107 functions and ~71 classes/types (regex estimate).

ACTIVITY



METRICS

METRIC	VALUE
Primary language	Python
Total LoC	7,184 across 42 files
Merged PRs / Commits	7 / 36
PR mix (Simple/Standard/Rich)	0.0% / 28.6% / 71.4%
Avg / median LoC per PR	358 / 103
% PRs referencing an issue	100.0%
Test-to-code / untested files	2.2% / 100.0%
Functions / Classes (est.)	107 / 71
Contributors	2
History	2026-05-30 → 2026-08-12
License	copyleft
CI / reproducible build	.github/workflows / none

REPRESENTATIVE CODE

tradesim-server/app/broker.py — Python, lines 246–285 of 333

```
def _place_market(self, product_id: str, side: str,
                  quote_size: Optional[str] = None, base_size: Optional[str] = None) -> str:
    coid = uuid.uuid4().hex
    if side == "BUY":
        resp = self._client.market_order_buy(client_order_id=coid, product_id=product_id, quote_size=quote_size)
    else:
        resp = self._client.market_order_sell(client_order_id=coid, product_id=product_id, base_size=base_size)

    data = resp if isinstance(resp, dict) else getattr(resp, "__dict__", {})
    success = data.get("success", getattr(resp, "success", False))
    if not success:
        raise BrokerError(f"Coinbase order rejected: {data}")
    # order id can live in a couple of shapes depending on SDK version
    sr = data.get("success_response") or getattr(resp, "success_response", None) or {}
    if isinstance(sr, dict):
        oid = sr.get("order_id")
    else:
        oid = getattr(sr, "order_id", None)
    oid = oid or data.get("order_id") or getattr(resp, "order_id", None)
    if not oid:
        raise BrokerError(f"Coinbase order accepted but no order_id returned: {data}")
    return oid

def _fill(self, order_id: str, attempts: int = 6, delay: float = 1.0) -> dict:
    """Poll the order until filled; return normalized fill numbers."""
    last = {}
    for _ in range(attempts):
        resp = self._client.get_order(order_id=order_id)
        order = resp.get("order") if isinstance(resp, dict) else getattr(resp, "order", None)
        order = order or (resp if isinstance(resp, dict) else getattr(resp, "__dict__", {}))
        status = _g(order, "status")
        filled_size = _f(_g(order, "filled_size"))
        filled_value = _f(_g(order, "filled_value"))
        fees = _f(_g(order, "total_fees"))
        avg_price = _f(_g(order, "average_filled_price"))
        last = {
            "status": status, "filled_size": filled_size, "filled_value": filled_value,
            "fees": fees, "avg_price": avg_price,
        }
        if status in {"FILLED", "DONE"} and filled_size > 0:
```

tradesim-server/app/db.py — Python, lines 147–186 of 267

```

def _run_migrations() -> None:
    """Idempotently add columns introduced after a DB was first created.
    create_all() makes new tables but never alters existing ones."""
    from sqlalchemy import inspect, text
    insp = inspect(ENGINE)
    adds = {
        "settings": [
            ("audit_enabled", "BOOLEAN DEFAULT FALSE"),
            ("interval_minutes", "INTEGER DEFAULT 15"),
            ("prev_rec_sig", "VARCHAR(96)"),
            ("min_hold_hours", "INTEGER DEFAULT 6"),
            ("history_since", "TIMESTAMP"),
            ("veto_exclusion_hours", "INTEGER DEFAULT 24"),
        ],
        "portfolio": [
            ("pos_opened_at", "TIMESTAMP"),
            ("last_change_at", "TIMESTAMP"),
            ("pos_peak_price", "FLOAT DEFAULT 0"),
        ],
        "trades": [
            ("fee_usd", "FLOAT DEFAULT 0"),
        ],
        "scans": [
            ("candidates_json", "TEXT"),
        ],
    }
    for table, cols_to_add in adds.items():
        try:
            existing = {c["name"] for c in insp.get_columns(table)}
        except Exception:
            continue # table not created yet; create_all handles it
        for name, ddl in cols_to_add:
            if name not in existing:
                with ENGINE.begin() as conn:
                    conn.execute(text(f"ALTER TABLE {table} ADD COLUMN {name} {ddl}"))

def init_db() -> None:
    """Create tables and seed the singleton control rows if missing."""
    Base.metadata.create_all(ENGINE)

```

```

def feat(closes: List[float], vols: List[float], i: int) -> Optional[Feat]:
    if i < WARMUP:
        return None
    seq = closes[: i + 1]
    price = seq[-1]
    if price <= 0:
        return None
    ssma = signals.sma(seq, SHORT) or 0.0
    lsma = signals.sma(seq, LONG) or 0.0
    rsi = signals.rsi(seq, RSI_P)
    ret24 = (price / seq[-25] - 1) * 100 if len(seq) > 25 and seq[-25] > 0 else 0.0
    mom = (price / seq[-1 - LOOK] - 1) * 100 if len(seq) > LOOK and seq[-1 - LOOK] > 0 else 0.0
    ext = (price / ssma - 1) * 100 if ssma > 0 else 0.0
    bo_bars = config.BREAKOUT_ROC_BARS
    roc3 = (price / seq[-1 - bo_bars] - 1) * 100 if len(seq) > bo_bars and seq[-1 - bo_bars] > 0 else 0.0
    vseq = vols[: i + 1]
    vsurge = 0.0
    if len(vseq) >= 4:
        recent = max(vseq[-1], vseq[-2])
        baseline = vseq[-26:-2] if len(vseq) >= 26 else vseq[: -2]
        avg = sum(baseline) / len(baseline) if baseline else 0.0
        vsurge = recent / avg if avg > 0 else 0.0
    return Feat(price, ret24, mom, rsi, ssma, lsma, ssma > lsma, ext, roc3, vsurge)

# ----- strategies: each returns the target base (or None = cash) -----
# A strategy may instead return (target, strong): strong=True marks a breakout-
# grade signal that skips the 2-scan confirmation (mirrors the live engine).

def strat_current(feats: Dict[str, Feat], holding: Optional[str] = None) -> Optional[str]:
    # candidate pool = top 24h gainers, then momentum edge (mirrors live logic)
    pool = sorted(feats.items(), key=lambda kv: kv[1].ret24, reverse=True)[:12]
    best, best_edge = None, 0.0
    for b, f in pool:
        edge = f.mom
        if not f.trend_up:
            edge *= 0.3
        if f.rsi is not None:
            if f.rsi >= 70:
                edge *= 0.4

```

SAMPLE PULL REQUESTS

Merge pull request #7 from jacobdmitch/claude/trading-engine-fees-decision-retnm

2026-07-26 · 7 files · +196/-24 · linked issue

```
diff --git a/TradeSim/Services/Predictor.swift b/TradeSim/Services/Predictor.swift
--- a/TradeSim/Services/Predictor.swift
+++ b/TradeSim/Services/Predictor.swift
@@ -10,4 +10,9 @@ struct Predictor {
    var strategy: StrategyConfig
    var config: RotationConfig
+   /// Minimum expected profit, net of round-trip fees, before a move is worth
+   /// making. Percent thresholds alone hide fee drag on a small balance: at
+   /// 0.6%/leg a $27 position pays ~$0.32 per round trip, so a 0.2% "edge" is
+   /// 5c of upside against 32c of cost.
+   var minNetProfitUSD: Double = 0.25

    // MARK: - Scoring
@@ -54,18 +59,49 @@ struct Predictor {
    // MARK: - Recommendation

+   /// Dollar cost/benefit of a pending action at the current account size.
+   ///
+   /// `feeLegs` is 2 for an entry or a rotation – the position has to be sold
+   /// again, so the decision owns the whole round trip – and 1 for an exit.
+   /// `edgePct` is the expected advantage of acting.
+   private func economics(value: Double, feeLegs: Int, edgePct: Double,
+                           feeRate: Double) -> (fee: Double, gross: Double, net: Double) {
+       let tradeValue = max(value, 0)
+       let fee = tradeValue * feeRate * Double(feeLegs)
+       let gross = tradeValue * edgePct / 100
+       return (fee, gross, gross - fee)
+   }

+   /// Decides what to do given the ranked scores and the current portfolio.
+   /// `feeRate` is the per-trade fee (e.g. 0.006); a rotation pays it twice.
+   ///
+   /// Every gain-seeking move has to clear its trading cost twice over: once in
+   /// percent (the thresholds plus the round trip) and once in dollars
+   /// (`minNetProfitUSD` of expected profit after fees), so an edge that
+   /// technically qualifies but only amounts to pennies never becomes a trade.
+   func recommend(ranked: [CoinScore], portfolio: Portfolio, feeRate: Double) -> RotationRecommendation {
+       let roundTripCostPct = feeRate * 2 * 100
+       let accountValue = portfolio.totalValue
+       let best = ranked.first

        // Currently in cash – look for a coin worth deploying into.
        guard let position = portfolio.position else {
            if let best, best.predictedEdgePct > config.enterThresholdPct + feeRate * 100 {
                // Entering commits to a round trip, so the edge must cover both legs.
                if let best, best.predictedEdgePct > config.enterThresholdPct + roundTripCostPct {
                    let e = economics(value: accountValue, feeLegs: 2,
                                        edgePct: best.predictedEdgePct, feeRate: feeRate)
                    if e.net < minNetProfitUSD {
                        return RotationRecommendation(
                            action: .hold, fromBase: nil, toBase: nil,
                            rationale: String(format: "%@'s %.1f%% edge is only %.2f on %.2f; $.2f of fees leaves $.2f, under the $.2f minimum. Not worth the trade – staying in cash.",
                                                best.base, best.predictedEdgePct, e.gross, accountValue,
                                                e.fee, e.net, minNetProfitUSD),
                            edgePct: best.predictedEdgePct, timestamp: Date())
                    }
                }
            }
            return RotationRecommendation(
                action: .enter, fromBase: nil, toBase: best.base,
                rationale: String(format: "%@ leads with a %.1f%% predicted edge (24h %@.1f%%).",
                                    best.base, best.predictedEdgePct, best.base, best.predictedEdgePct),
                rationale: String(format: "%@ leads with a %.1f%% predicted edge (24h %@.1f%%): ~$.2f on $.2f less $.2f round-trip fees = $.2f net."))
        }
    }
}
```

Merge pull request #6 from jacobdmitch/claude/trading-model-data-integrity-5kp431

2026-06-12 · 4 files · +35/-11 · linked issue

```
diff --git a/tradesim-server/app/auditor.py b/tradesim-server/app/auditor.py
--- a/tradesim-server/app/auditor.py
+++ b/tradesim-server/app/auditor.py
@@ -40,8 +40,14 @@ _SYSTEM = (
     " 1. Data integrity: prices/volume/momentum that look anomalous, stale, or "
     "internally inconsistent (e.g. a huge 'gain' on near-zero volume = thin-book "
     "artifact).\n"
-    " 2. Liquidity traps: the target coin is far too illiquid to enter/exit cleanly.\n"
-    " 3. Adverse news: use web search to check for very recent delisting, exchange "
-    "removal, hack/exploit, depeg, team/rug events, or trading halts for the coin.\n"
+    "artifact). NOTE: all volume figures are Coinbase-exchange-specific and will be "
+    "lower than global volume – do NOT veto solely because Coinbase volume is a "
+    "fraction of global volume. Only veto when Coinbase volume itself is suspiciously "
+    "tiny (e.g. under $50K/day) for an active action, or when the figure is "
+    "internally inconsistent with the price action shown.\n"
+    " 2. Liquidity traps: the target coin is far too illiquid on Coinbase to "
+    "enter/exit cleanly at the proposed position size.\n"
+    " 3. Adverse news: use web search to check for very recent delisting from "
+    "Coinbase, hack/exploit, depeg, token merger/restructuring (e.g. fixed-ratio "
+    "conversion to another token), team/rug events, or trading halts for the coin.\n"
+    "Approve by default. Only veto (downgrade to HOLD) when you find a concrete, "
+    "specific problem – never on vague caution. "
diff --git a/tradesim-server/app/config.py b/tradesim-server/app/config.py
--- a/tradesim-server/app/config.py
+++ b/tradesim-server/app/config.py
@@ -40,5 +40,5 @@ QUOTE_CURRENCY = "USD"
GRANULARITY = 3600 # 1-hour candles
FEE_RATE = 0.006 # Coinbase Advanced taker fee, low-volume tier (~0.6%/leg)
-MIN_LIQUIDITY_USD = 25_000 # minimum 24h USD volume to be a rotation candidate
+MIN_LIQUIDITY_USD = 100_000 # minimum 24h USD volume on Coinbase to be a rotation candidate

STABLECOINS = {
@@ -47,4 +47,10 @@ STABLECOINS = {
}

+# Tokens that have been merged, deprecated, or asset-restructured and must never be traded.
+# These are permanent exclusions beyond the dynamic veto system.
+DEPRECATED_TOKENS = {
+    "STG", # merged into LayerZero (ZRO) at fixed 0.08634 ratio, April 2026; Binance.US/Coinmetro delisted
+}
+
+@dataclass
diff --git a/tradesim-server/app/market.py b/tradesim-server/app/market.py
--- a/tradesim-server/app/market.py
+++ b/tradesim-server/app/market.py
@@ -50,5 +50,5 @@ class MarketStat:

def fetch_usd_products() -> List[Product]:
-    """All online, tradable USD products (stablecoins filtered out)."""
+    """All online, tradable USD products (stablecoins and deprecated tokens filtered out)."""
    r = _SESSION.get(f"{EXCHANGE}/products", timeout=_TIMEOUT)
    r.raise_for_status()
@@ -60,4 +60,5 @@ def fetch_usd_products() -> List[Product]:
    and not d.get("trading_disabled", False)
    and d.get("base_currency") not in config.STABLECOINS
+    and d.get("base_currency") not in config.DEPRECATED_TOKENS
+):
    out.append(
@@ -88,11 +89,17 @@ def fetch_stats(products: List[Product]) -> List[MarketStat]:
```

Merge pull request #5 from jacobdmitch/claude/token-exclusion-auditor-vetos-2TZ3j

2026-05-31 · 5 files · +94/-9 · linked issue

```

diff --git a/tradesim-server/app/config.py b/tradesim-server/app/config.py
--- a/tradesim-server/app/config.py
+++ b/tradesim-server/app/config.py
@@ -109,4 +109,7 @@ CRON_GRANULARITY_MIN = 15 # must match render.yaml cron schedule
    MIN_HOLD_HOURS_DEFAULT = int(os.environ.get("MIN_HOLD_HOURS", "6"))

+# Hours a token stays excluded after receiving 2 consecutive AI Auditor vetos.
+VETO_EXCLUSION_HOURS_DEFAULT = int(os.environ.get("VETO_EXCLUSION_HOURS", "24"))
+
+    COINBASE_API_KEY = os.environ.get("COINBASE_API_KEY", "").strip()
+    # Allow the PEM to be supplied with escaped newlines.
diff --git a/tradesim-server/app/db.py b/tradesim-server/app/db.py
--- a/tradesim-server/app/db.py
+++ b/tradesim-server/app/db.py
@@ -50,4 +50,5 @@ class Settings(Base):
    prev_rec_sig: Mapped[Optional[str]] = mapped_column(String(96), nullable=True) # 2-scan confirm
    min_hold_hours: Mapped[int] = mapped_column(Integer, default=6) # min hold before rotating
+    veto_exclusion_hours: Mapped[int] = mapped_column(Integer, default=24) # hours a token is banned after 2 consecutive
vetos
    # Dashboard only shows history (trades/recs/equity/scans/audits) at or after
    # this time; advanced on each deliberate DRY-RUN<->LIVE switch to start fresh.
@@ -154,4 +155,5 @@ def _run_migrations() -> None:
    ("min_hold_hours", "INTEGER DEFAULT 6"),
    ("history_since", "TIMESTAMP"),
+    ("veto_exclusion_hours", "INTEGER DEFAULT 24"),
+
    ],
    "portfolio": [
@@ -205,4 +207,33 @@ def get_portfolio(session) -> Portfolio:

+def get_veto_excluded_bases(session, exclusion_hours: int) -> dict:
+    """Return {base: expires_at} for tokens with 2 consecutive AI Auditor vetos
+    whose most recent veto falls within the exclusion window."""
+    from datetime import timedelta
+    if exclusion_hours <= 0:
+        return {}
+    now = _now()
+    cutoff = now - timedelta(hours=exclusion_hours)
+    bases = [b for (b,) in session.query(AuditLog.to_base).filter(
+        AuditLog.to_base.isnot(None)
+    ).distinct().all()]
+    result = {}
+    for base in bases:
+        recent = (
+            session.query(AuditLog)
+            .filter(AuditLog.to_base == base)
+            .order_by(AuditLog.id.desc())
+            .limit(2)
+            .all()
+        )
+        if len(recent) == 2 and all(a.verdict == "VETO" for a in recent):
+            latest_ts = recent[0].ts
+            if latest_ts.tzinfo is None:
+                latest_ts = latest_ts.replace(tzinfo=timezone.utc)
+            if latest_ts >= cutoff:
+                result[base] = latest_ts + timedelta(hours=exclusion_hours)
+    return result
+
+def reset_for_mode_switch(session, *, to_live: bool) -> None:
+    """Re-baseline the account when the operator deliberately switches between

```

HYGIENE

Clean. No secret patterns or PII keywords in tracked files.

Contains real code excerpts and diff hunks (secret-shaped values redacted). % Rich PRs is a lower bound — review threads live on the code host, not in git. Function and class counts are regex estimates.